Experiment 13

Case Study on Limitations of Low-Code Platforms for a Complex Enterprise Use Case

Name: Hoshni Sivaprakash | **Register Number:** 241401029

Aim

To study the limitations of low-code platforms when they are applied to complex enterprise-level applications and to understand when pro-code extensions or hybrid approaches may become necessary.

Introduction

Low-code platforms allow applications to be developed mainly through visual development tools instead of writing large amounts of traditional program code. Applications are assembled using predefined components such as forms, reports and dashboards, and behaviour is defined through workflows, rules and configuration. A small amount of scripting is usually available for specific needs. This approach reduces development time and effort and makes it possible for a wider range of people to take part in building business applications.

Low-code platforms are well suited to standard business applications. As the requirements of an enterprise grow in terms of users, data volume, process complexity, integrations, security and customisation, the built-in capabilities of a platform may no longer be sufficient on their own. This case study examines such limitations through an enterprise student admission scenario.

Case Study: Enterprise Student Admission Management System

An educational institution requires a system to manage its student admission operations. The system may need to serve the following users and handle the following functions:

Category	Items
Users	Applicants and parents, admission staff, admission managers, department heads and administrators
Data and operations	Applicant records, admission enquiries, course details, enquiry follow-up and admission history
Supporting functions	Notifications, fee payments and document verification
Enterprise integration and insight	Student information system / ERP integration, analytics and reporting

Figure 1 shows the architecture of this system on a low-code platform. The architecture is arranged as six groups that are connected from left to right:

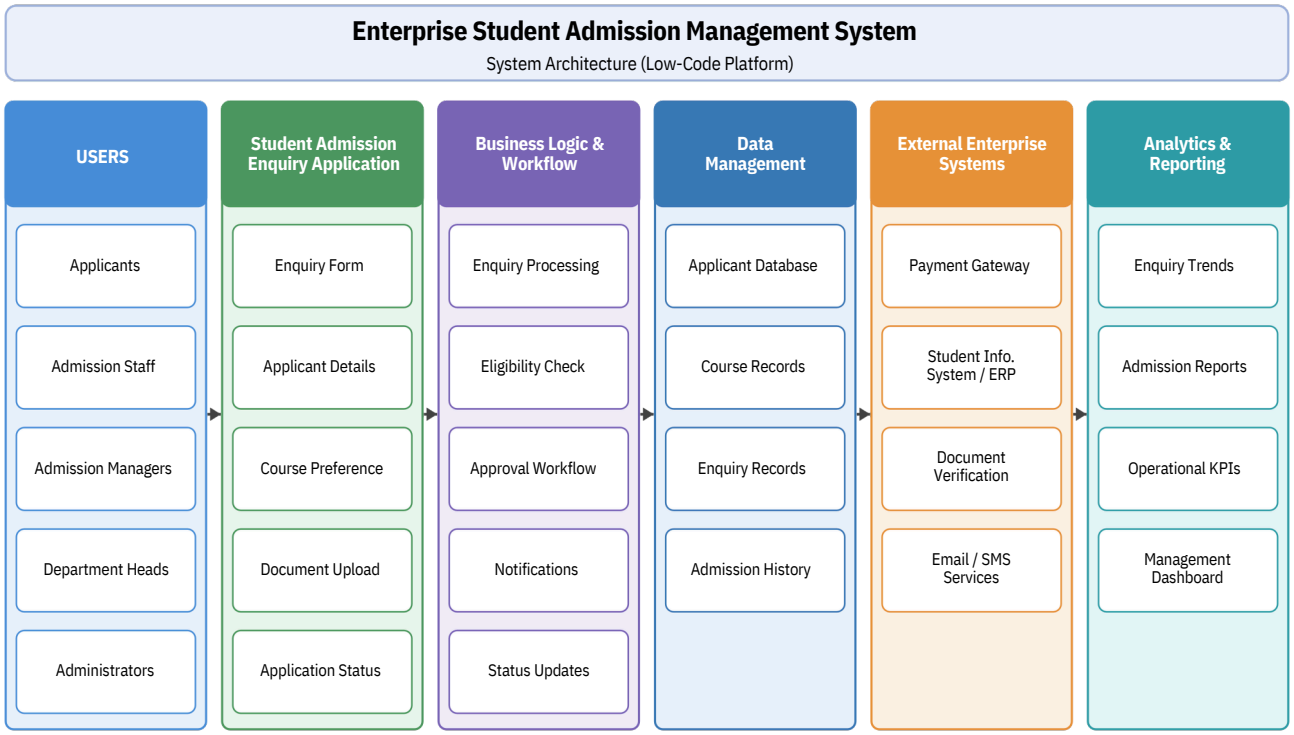


Figure 1: Enterprise Student Admission Management System Architecture

- **Users:** applicants, admission staff, admission managers, department heads and administrators, who interact with the system.
- **Student Admission Enquiry Application:** the user-facing part, covering the enquiry form, applicant details, course preference, document upload and application status.
- **Business Logic & Workflow:** enquiry processing, eligibility check, approval workflow, notifications and status updates, which move an enquiry through the process.
- **Data Management:** the applicant database, course records, enquiry records and admission history, in which the information is stored.
- **External Enterprise Systems:** the payment gateway, student information system / ERP, document verification and email / SMS services with which the application must exchange information.
- **Analytics & Reporting:** enquiry trends, admission reports, operational KPIs and a management dashboard, which present the stored and processed data to decision makers.

The groups interact in sequence. Users enter and view information through the application; the business logic and workflow layer processes this information; the data layer stores and supplies it; external systems provide or receive related information such as fee payments, student records and document checks; and the analytics layer reports on the results.

How a Low-Code Platform Can Support the System

A large part of the system consists of standard application functionality that low-code platforms are designed to handle:

Capability	Use in the Student Admission System
Forms	Capturing admission enquiries, applicant details and course preferences.
Reports	Viewing enquiries, admission history and applicant reports.
Workflows	Moving an enquiry through processing, approval, assignment and status updates.
Business process automation	Applying rules and automatic actions, such as updating a status when a step is completed.
Data management	Storing applicant, course and enquiry records and performing standard create, read, update and delete operations.
User interfaces	Providing web and mobile screens for applicants, staff, managers and administrators.
Notifications	Informing applicants about enquiry status through email or similar messages.
Basic integrations	Connecting to common services through built-in connectors or standard interfaces.

Limitations at Enterprise Scale

As the system grows into a complex enterprise application, the following limitations may become significant. They are discussed individually below.

Performance and Scalability

When the number of enquiries, applicant records and users becomes very large, especially during peak admission seasons, performance may be affected. Complex queries and reports over large amounts of data can respond slowly, and the platform may impose limits on how far an application can be scaled. Because the underlying infrastructure is managed by the platform, the options for tuning performance are limited.

Complex Business Logic

Highly specialised business rules, such as intricate eligibility and approval chains, rank-based or merit-based seat allocation across courses and advanced processing, may be difficult to express using visual workflows and standard scripting. Such requirements may need custom development.

System Integration

Connecting the application to student information systems, ERP and CRM systems, payment gateways, legacy systems, external APIs and other enterprise services can be challenging. Built-in connectors may not cover every system or data format, and complex or real-time integrations may need custom interfaces.

Customization

Standard low-code components provide common screens and layouts. Highly specialised interfaces and user experiences, or functionality that is not offered by the platform, may exceed what these standard components can provide.

Security and Governance

Enterprises require detailed access control, governance, compliance and strong security practices. Role- and permission-based controls offered by a platform may need to be supplemented to meet fine-grained, regulatory or organisation-specific requirements, particularly for applicants' personal data, and governance of many applications must be managed carefully.

AI and Advanced Technology

Specialised artificial intelligence and machine-learning models, such as predicting admission demand or applicant interest, and other advanced processing may require external services or custom code, since a low-code platform may not provide such capabilities directly.

Vendor / Platform Dependency

The application depends on the capabilities, restrictions, pricing and supported integrations of the selected platform. Features may be limited by the plan or edition, and moving an application to another platform later can be difficult.

Data Management

Large datasets, complex relationships between records and advanced data processing, such as heavy batch processing or complex data transformations, may be difficult to handle within the data model and tools provided by the platform.

Figure 2 summarises how the limitations arise. A basic low-code application, characterised by quick development, minimal configuration and suitability for small and medium needs, faces growing enterprise requirements: more users and data, complex business processes, integration with multiple systems and higher security and compliance needs. These lead to enterprise-scale challenges such as increased complexity, higher performance demands, greater integration needs, stronger security and governance, and more customisation and flexibility. The diagram then identifies eight key limitations, which are the ones discussed above.

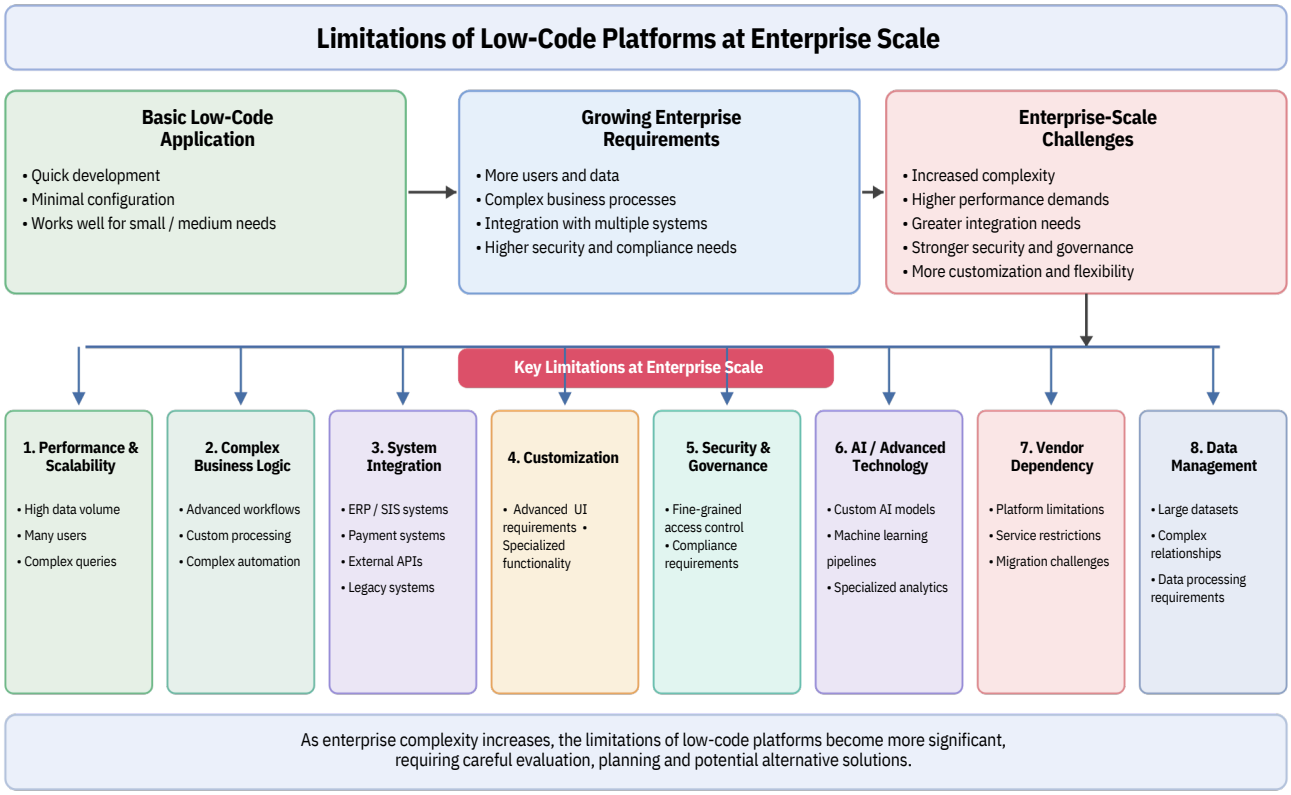


Figure 2: Limitations of Low-Code Platforms at Enterprise Scale

The relevance of each limitation to the Enterprise Student Admission Management System is given below.

Limitation	Relevance to the Case Study
Performance & Scalability	Growing numbers of applicants and enquiries, especially in peak admission seasons, and reports over admission history.
Complex Business Logic	Eligibility checks, approval workflows and seat-allocation rules that become more complex over time.
System Integration	Payment gateway, student information system / ERP, document verification and email / SMS services.
Customization	Specialised interfaces for staff, managers and department heads, and a tailored applicant experience.
Security & Governance	Different access needs for applicants, admission staff, managers and administrators; protection of applicants' personal and payment-related data.
AI / Advanced Technology	Possible intelligent features built on enquiry history and admission data.
Vendor Dependency	Reliance on the platform for all application, workflow and reporting features, and on its plan limits.
Data Management	Large volumes of applicant records and admission history with relationships to courses and enquiries.

Low-Code vs Pro-Code Requirements

The suitability of low-code or pro-code development depends on the requirements of the application. Neither approach is universally better; the comparison below indicates the typical role of each.

Area	Low-Code Approach	When Pro-Code May Be Required
User interface	Standard screens, forms and layouts built from predefined components.	Highly specialised interfaces or custom user experiences beyond standard components.
Business logic	Visual workflows, rules and simple scripting for common business processes.	Complex rules and advanced processing that cannot be expressed in visual tools.
Data management	Built-in data model with forms, relationships and standard create, read, update and delete operations.	Very large datasets, complex relationships and advanced data processing.
Integrations	Built-in connectors and standard interfaces for common services.	Legacy systems, specialised or real-time integrations and systems without connectors.
Automation	Rules, scheduled actions and workflow-based automation.	Complex or high-volume automation and specialised processing.
AI/ML	Basic features offered by the platform, where available.	Custom AI models, machine-learning pipelines and specialised analytics.
Security	Role- and permission-based access control provided by the platform.	Fine-grained access control, specific compliance requirements and enterprise security needs beyond platform features.
Scalability	Suitable where the platform handles the expected users and data volumes.	High data volumes, large user numbers and demanding performance requirements.
Custom functionality	Functionality available as platform features.	Functionality that the platform does not provide.

Hybrid Approach

Organisations can combine low-code development with pro-code extensions. Low-code is used to develop standard application functionality quickly, while custom code, APIs, integrations or specialised processing are used for requirements that exceed the built-in capabilities of the platform. The structure consists of three layers:

Low-Code Application → Pro-Code Extensions → Enterprise Systems

Figure 3 shows this architecture. Users such as applicants, admission staff, admission managers, department heads and administrators access the system at the top, and the three layers are described below.

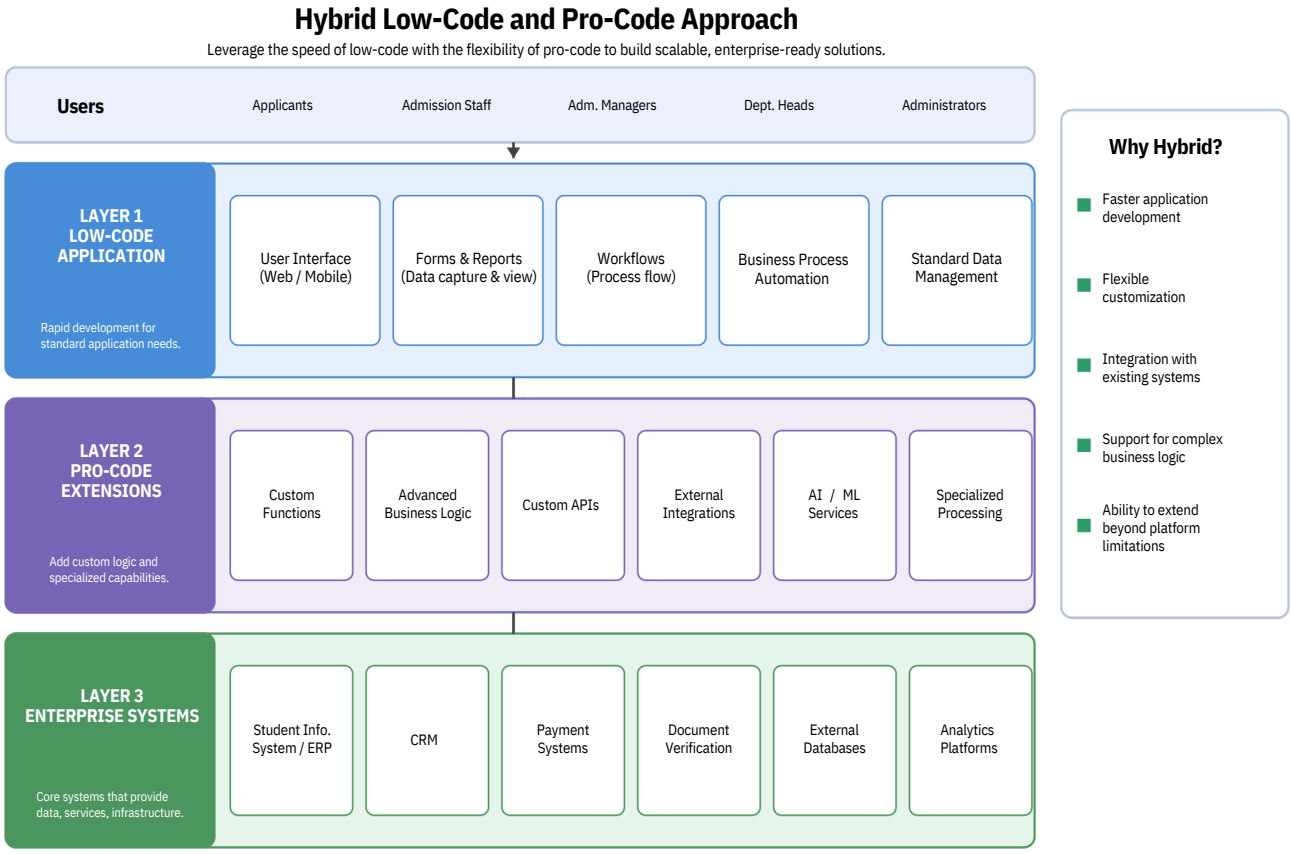


Figure 3: Hybrid Low-Code and Pro-Code Enterprise Architecture

Layer	Role
Layer 1 – Low-Code Application	Provides rapid development for standard application needs: user interface (web / mobile), forms and reports, workflows, business process automation and standard data management.
Layer 2 – Pro-Code Extensions	Adds custom logic and specialised capabilities: custom functions, advanced business logic, custom APIs, external integrations, AI / machine-learning services and specialised processing.
Layer 3 – Enterprise Systems	Core systems that provide data, services and infrastructure: student information system / ERP, CRM, payment systems, document verification, external databases and analytics platforms.

The diagram also lists the reasons for a hybrid approach: faster application development, flexible customisation, integration with existing systems, support for complex business logic and the ability to extend beyond platform limitations. The arrows between the layers indicate that information flows in both

directions between the application, its extensions and the enterprise systems.

When Pro-Code Extension May Become Necessary

- **Complex business logic:** when business rules and processing cannot be represented using standard workflows and rules.
- **Specialized integrations:** when the application must connect to legacy, ERP or other systems for which suitable connectors are not available.
- **Custom APIs:** when internal services or interfaces have to be exposed or consumed in a way that the platform does not support directly.
- **Advanced AI/ML:** when intelligent features require custom models or machine-learning pipelines.
- **High-performance processing:** when large batches of data or real-time processing exceed what the platform can handle efficiently.
- **Specialized user interfaces:** when the required user experience goes beyond standard components.
- **Complex enterprise data processing:** when large datasets and complex relationships require advanced data handling.
- **Requirements beyond platform capabilities:** when any needed functionality is simply not offered by the selected platform.

Case Study Analysis

For the Enterprise Student Admission Management System, a low-code platform can be used effectively to develop the standard parts of the system rapidly: forms for admission enquiries, applicant and course details, reports on admission history, workflows for enquiry processing and approval, notifications, and basic dashboards. These features correspond to the capabilities that low-code platforms are designed to provide. The Student Admission Enquiry Form application built in Zoho Creator in the earlier experiments reflects this, using a form, a form workflow that notifies the student, and role-based access with Admission Manager and Admission Staff roles.

As the institution grows, the requirements become more demanding. A large number of applicants, enquiries and admission records raises performance and data-management concerns; integration with payment, student information and document verification systems raises integration concerns; differing access needs and the handling of applicants' personal data raise security and governance concerns; and advanced features such as intelligent analysis may need external services. In such cases, extensions, integrations or custom development may be needed in addition to the low-code application. Low-code is therefore useful for rapidly developing standard business applications, whereas increasingly complex enterprise requirements may call for a hybrid approach, and the choice should be based on the requirements of the application.

Result

The case study demonstrated the major limitations that can arise when low-code platforms are applied to complex enterprise requirements, namely performance and scalability, complex business logic, system integration, customisation, security and governance, AI and advanced technology, vendor or platform dependency, and data management. It also showed how a hybrid low-code/pro-code approach can address requirements beyond standard low-code capabilities.

Conclusion

Low-code platforms offer speed and simplicity for standard business applications, but their limitations become more significant as enterprise complexity increases. For an application such as the Enterprise Student Admission Management System, standard functionality can be built quickly using low-code, while demanding requirements in logic, integration, security, scale and advanced technology may need pro-code extensions. It is therefore important to understand the limitations of a platform before selecting a development approach, so that the chosen approach matches the requirements of the enterprise application.

24140107 9